1. Preflight.py adalah pemeriksaan awal sebelum aplikasi dijalankan. Tujuannya memastikan lingkungan dan file-file penting sudah siap. Mengimpor Path → untuk mengecek file/folder. sqlite3 → untuk mengetahui versi SQLite yang tersedia. sys → untuk mengetahui versi Python dan mengatur exit code. Mengecek ada tidaknya file input users.json','products.json','transactions.json . Mengecek apakah app bisa di-import “try: import app print('PASS import app') except Exception as e: print('FAIL import app',e) F.append('import')”. Menentukan apakah preflight berhasil “print('PREFLIGHT READY' if not F else 'PREFLIGHT FAILED')”. Memberikan exit code “raise SystemExit(bool(F))”

```
PS C:\Users\nurochman.MAMNB-0420\Downloads\FSB_Sesi_3_Pipeline_Engineering_Project\FSB_Sesi_3_Pipeline_Engineering_Proje
ct> python preflight.py
Python 3.12.0
SQLite 3.42.0
PASS data\raw\users.json
PASS data\raw\products.json
PASS data\raw\transactions.json
PASS import app
PREFLIGHT READY
```

2. quality_checks/01_environment.py **mengecek dan menampilkan versi Python dan SQLite yang sedang digunakan”** import sqlite3, sys print("PASS", sys.version.split()[0], sqlite3.sqlite_version)”. Pemeriksaan berhasil. Python yang digunakan adalah **3.12.0**, dan SQLite yang tersedia adalah **3.42.0**.

```
PS C:\Users\nurochman.MAMNB-0420\Downloads\FSB_Sesi_3_Pipeline_Engineering_Project\FSB_Sesi_3_Pipeline_Engineering_Project> python quality_checks/01_environment.py
PASS 3.12.0 3.42.0
```

3. quality_checks/02_cli_contract.py untuk menguji apakah command **`app ingest --help`** tersedia dan memiliki opsi CLI yang wajib**, yaitu `--raw`, `--db`, dan `--reset`. Menjalankan comman “**r = subprocess.run([sys.executable, "-m", "app", "ingest", "--help"], text=True, capture_output=True)”. Mengecek hasil “ok = (r.returncode == 0 and "--raw" in r.stdout and "--db" in r.stdout and "--reset" in r.stdout)”. Script menyatakan **PASS** hanya jika semua kondisi di atas terpenuhi. Jika Exit code 0 maka PASS. Jika Exit code 1 maka FAIL

```
PS C:\Users\nurochman.MAMNB-0420\Downloads\FSB_Sesi_3_Pipeline_Engineering_Project\FSB_Sesi_3_Pipeline_Engineering_Project> python quality_checks/02_cli_contract.py
PASS 0 usage: python -m app ingest [-h] --raw RAW --db DB [--reset] [--output OUTPUT]
                           [--log-level {DEBUG,INFO,WARNING,ERROR}]
                           [--log-file LOG_FILE]

options:
  -h, --help            show this help message and exit
  --raw RAW
  --db DB
  --reset
  --output OUTPUT
  --log-level {DEBUG,INFO,WARNING,ERROR}
  --log-file LOG_FILE
```

4. quality_checks/03_happy_path.py bertujuan melakukan integration test untuk proses **`ingest`:** menjalankan aplikasi untuk memasukkan data mentah dari `data/raw` ke database SQLite baru, lalu memastikan jumlah data yang masuk persis sesuai yang diharapkan**.** Alurnya


```
PS C:\Users\nurochman.MAMNB-0420\Downloads\FSB_Sesi_3_Pipeline_Engineering_Project\FSB_Sesi_3_Pipeline_Engineering_Project> python quality_checks/03_happy_path.py
PASS 0 {'users': 14, 'products': 10, 'transactions': 22} 2026-09-04 14:10:23,225 | INFO | app.cli | step=start raw=data/raw db=data\output\c3\app.db reset=True
2026-09-04 14:10:23,256 | INFO | app.cli | step=extract entity=users rows=16
2026-09-04 14:10:23,256 | INFO | app.cli | step=extract entity=products rows=12
2026-09-04 14:10:23,256 | INFO | app.cli | step=extract entity=transactions rows=25
2026-09-04 14:10:23,256 | INFO | app.cli | step=clean entity=users raw=16 clean=14 rejected=2
2026-09-04 14:10:23,256 | INFO | app.cli | step=clean entity=products raw=12 clean=10 rejected=2
2026-09-04 14:10:23,256 | INFO | app.cli | step=clean entity=transactions raw=25 clean=25 rejected=0
2026-09-04 14:10:23,287 | INFO | app.cli | step=verify expected={'users': 14, 'products': 10, 'transactions': 22} actual={'users': 14, 'products': 10, 'transactions': 22} out
come=pass
2026-09-04 14:10:23,287 | WARNING | app.cli | step=quarantine rejected=7 by_entity={'users': 2, 'products': 2, 'transactions': 3}
2026-09-04 14:10:23,287 | INFO | app.cli | step=load users=14 products=10 transactions=22 rejected=7
2026-09-04 14:10:23,287 | INFO | app.cli | step=complete outcome=success
```

5. quality_checks/04_logging.py menguji apakah proses `ingest` berjalan melalui semua tahapan ETL dan mencatat setiap tahap tersebut ke file log**.**

6. quality_checks/05_error_policy.py untuk menguji apakah aplikasi **app ingest** menangani beberapa kondisi error dengan benar (error handling)**.** Fungsi ini dibuat agar pengujian tiga skenario tidak perlu menulis kode yang sama berulang-ulang(def run(name)). Membuat folder output terpisah “out=Path("data/output")/f"c5_{name}". data/output/c5_missing_users/, data/output/c5_wrong_field/, data/output/c5_empty_transactions/. Menghapus database lama “db=out/"app.db", “db.unlink(missing_ok=True)”. Menjalankan app ingest. Menjalankan tiga skenario error “a=run("missing_users"), b=run("wrong_field"), c=run("empty_transactions")”. Memastikan error code sesuai desain aplikasi “ok=(a.returncode,b.returncode,c.returncode) ==(2,3,0). Menampilkan pesan error dan Exit code untuk automation (raise SystemExit(not ok)

```
PS C:\Users\nurochman.MAMNB-0420\Downloads\FSB_Sesi_3_Pipeline_Engineering_Project\FSB_Sesi_3_Pipeline_Engineering_Proje
ct> python quality_checks/05_error_policy.py
PASS 2 3 0 2026-09-05 12:35:08,051 | INFO | app.cli | step=start raw=data/error_fixtures/missing_users db=data\output\c5
_missing_users\app.db reset=True
2026-09-05 12:35:08,051 | ERROR | app.cli | step=extract outcome=abort error=file_missing path=data\error_fixtures\missi
ng_users\users.json
 2026-09-05 12:35:08,145 | INFO | app.cli | step=start raw=data/error_fixtures/wrong_field db=data\output\c5_wrong_field
\app.db reset=True
2026-09-05 12:35:08,160 | ERROR | app.cli | step=validate outcome=abort error=missing_fields entity=users index=0 fields
=['user_id']
 2026-09-05 12:35:08,270 | INFO | app.cli | step=start raw=data/error_fixtures/empty_transactions db=data\output\c5_empt
y_transactions\app.db reset=True
2026-09-05 12:35:08,285 | INFO | app.cli | step=extract entity=users rows=16
2026-09-05 12:35:08,285 | INFO | app.cli | step=extract entity=products rows=12
2026-09-05 12:35:08,285 | INFO | app.cli | step=extract entity=transactions rows=0
2026-09-05 12:35:08,285 | WARNING | app.cli | step=extract entity=transactions rows=0 policy=continue
2026-09-05 12:35:08,285 | INFO | app.cli | step=clean entity=users raw=16 clean=14 rejected=2
2026-09-05 12:35:08,285 | INFO | app.cli | step=clean entity=products raw=12 clean=10 rejected=2
2026-09-05 12:35:08,285 | INFO | app.cli | step=clean entity=transactions raw=0 clean=0 rejected=0
2026-09-05 12:35:08,317 | INFO | app.cli | step=verify expected={'users': 14, 'products': 10, 'transactions': 0} actual=
{'users': 14, 'products': 10, 'transactions': 0} outcome=pass
2026-09-05 12:35:08,317 | WARNING | app.cli | step=quarantine rejected=4 by_entity={'users': 2, 'products': 2, 'transact
ions': 0}
```

7. python run_pipeline.py menjalankan proses ingest utama (sebagai pipeline runner) dan meneruskan status keberhasilannya sebagai exit code (bukan sebagai penguji hasil) . Membuat command “cmd=[sys.executable, '-m','app', 'ingest', '--raw','data/raw', '--db','data/output/app.db', '--reset', '--output','data/output', '--log-file','data/output/run.log']”. Menjalankan command tersebut “subprocess.run(cmd)”. Mengambil return code “subprocess.run(cmd).returncode”. Meneruskan status tersebut” raise SystemExit(subprocess.run(cmd).returncode)”

```
PS C:\Users\nurochman.MAMNB-0420\Downloads\FSB_Sesi_3_Pipeline_Engineering_Project\FSB_Sesi_3_Pipeline_Engineering_Proje
ct> python run_pipeline.py
2026-09-05 12:51:12,421 | INFO | app.cli | step=start raw=data/raw db=data/output/app.db reset=True
2026-09-05 12:51:12,436 | INFO | app.cli | step=extract entity=users rows=16
2026-09-05 12:51:12,436 | INFO | app.cli | step=extract entity=products rows=12
2026-09-05 12:51:12,436 | INFO | app.cli | step=extract entity=transactions rows=25
2026-09-05 12:51:12,436 | INFO | app.cli | step=clean entity=users raw=16 clean=14 rejected=2
2026-09-05 12:51:12,436 | INFO | app.cli | step=clean entity=products raw=12 clean=10 rejected=2
2026-09-05 12:51:12,436 | INFO | app.cli | step=clean entity=transactions raw=25 clean=25 rejected=0
2026-09-05 12:51:12,471 | INFO | app.cli | step=verify expected={'users': 14, 'products': 10, 'transactions': 22} actual
={'users': 14, 'products': 10, 'transactions': 22} outcome=pass
2026-09-05 12:51:12,471 | WARNING | app.cli | step=quarantine rejected=7 by_entity={'users': 2, 'products': 2, 'transact
ions': 3}
2026-09-05 12:51:12,471 | INFO | app.cli | step=load users=14 products=10 transactions=22 rejected=7
2026-09-05 12:51:12,471 | INFO | app.cli | step=complete outcome=success
```

8. validate_delivery.py untuk final verification (quality check) dan memeriksa apakah hasil pipeline sudah lengkap, benar, dan siap direview**.** Memastikan database tersedia “ if not db.exists(): print('FAIL app.db missing') raise SystemExit(1)”.Kalau tidak ada maka FAIL. Memeriksa jumlah data “counts={ n:c.execute(f'SELECT COUNT(*) FROM {n}').fetchone()[0] for n in ('users','products','transactions') }”. Kemudian dibandingkan dengan jumlah yang diharapkan “counts == { 'users':14, 'products':10, 'transactions':22 }” . Kalau sesuai → PASS tp kalau

berbeda → FAIL. Memeriksa foreign key “fks=len(c.execute('PRAGMA foreign_key_list(transactions)').fetchall())”. Memeriksa summary.json “s=json.loads((out/'summary.json').read_text()) if (out/'summary.json').exists() else {}”. Memeriksa `rejected_records.json`. Memeriksa kelengkapan log. Mengumpulkan semua kegagalan. Menentukan apakah pipeline siap direview

```
PS C:\Users\nurochman.MAMNB-0420\Downloads\FSB_Sesi_3_Pipeline_Engineering_Project\FSB_Sesi_3_Pipeline_Engineering_Project> python validate_delivery.py
PASS counts {'users': 14, 'products': 10, 'transactions': 22}
PASS foreign_keys 2
PASS summary
PASS rejected
PASS log

READY FOR REVIEW
PS C:\Users\nurochman.MAMNB-0420\Downloads\FSB_Sesi_3_Pipeline_Engineering_Project\FSB_Sesi_3_Pipeline_Engineering_Project>
```

KESIMPULAN dari **semua script yang kita bahas** adalah bahwa project tersebut membentuk sebuah **pipeline ETL/data ingestion yang dilengkapi automated testing dan final validation**.

Kalau ini adalah sebuah **project data engineering**, maka script-script tersebut menunjukkan pola: **Build → Run → Test → Validate → Accept**

Bukan sekadar:

"Saya berhasil memasukkan data ke database."

Tetapi:

"Saya bisa membuktikan bahwa pipeline ingestion berjalan, hasil datanya benar, error ditangani sesuai aturan, relasi database benar, rejected data terdokumentasi, dan seluruh proses tercatat di log."